CODING RULES: A GUIDE TO RSTUDIO

1. FILE NAMING & PROJECT STRUCTURE

- Naming Rules
  - The exact same naming rules apply to files, folders, AND project names.
  - Safe to use: Letters, numbers, dashes, and underscores.
  - Avoid entirely: Spaces, parentheses, brackets, colons, slashes, commas, and symbols like @, $, &, *, !
  - Why it matters: If a file path contains an illegal character, Quarto will crash during rendering.
- Project Anchor Rule
  - Add a .Rproj file to your project folder. This is the RStudio anchor.
    - File -> New Project -> New Directory -> New Project (This automatically creates your folder and your .Rproj file).
    - The file name should match the folder name it sits inside.
  - Add a text file named '_quarto.yml' to your project folder. This is the Quarto anchor.
  - Why it matters: These anchors avoid a PermissionDenied error by creating a boundary line, eliminating the risk of Quarto wandering off and crashing.
- Daily Workflow Routine
  - Don't open files from Finder, and don't use Session -> Set Working Directory.
    - These methods are tedious and error-prone. They can break your relative paths and cause Quarto to fail to find your files.
  - Open RStudio first, click the project dropdown menu in the top-right corner, and teleport into your project.
  - Check the top-right corner to ensure it doesn't say Project: (None).
  - Open your files and render using Relative Paths (e.g., **image.png** instead of **/Users/hannah/Desktop/...**). RStudio will automatically remember your open tabs next time you return!

    - Because the project locks RStudio into the project folder, you never have to type out long computer paths like **/Users/hannahrozenbaum/Desktop/...**. Instead, everything becomes **relative** to where the **.Rproj** file sits.
    - Opening multiple files:
        - Use the Files pane in bottom right, and click the checkboxes of files you want to open. Click the blue gear icon and select "Open Selected Files".
        - To open all files within the project folder, check the first file, then hold down Shift and check the last file. Click the blue gear icon and select "Open Selected Files".

---

2. RSTUDIO FILE TECHNICAL SETUP

- YAML Headers
    - Must have perfect indentation (spaces only, no tabs).
    - Must always put a space after a colon.
    - At the very top of the file, enclosed by three dashes on either end.
- Troubleshooting
    - If theme compilation fails mysteriously, try disabling the Sass cache by running 'export QUARTO_SASS_CACHE=FALSE' in your terminal.
    - If the file fails to render, type 'quarto check' in the terminal to do a system health check.

---

3. ENVIRONMENT SETUP

- Package Loading
    - Always check if packages are loaded at the top of the file if an object/function is "not found".
    - Best Practice: Keep all library() calls inside a single R code chunk at the very top of your document so you can easily verify what's active.
- **Never Save Workspace** Rule
    - Change RStudio settings to never save workspace on exit, and never load .RData on startup.

  - Why it matters: This prevents conflicts between your current session and previous sessions, which can cause errors.

- R Session Restarts
  - Periodically restart your R session to clear the environment and avoid conflicts.
  - Shortcut: **Cmd + Shift + 0**

- Conflict Resolution
  - Be aware that some libraries share function names. Use the syntax **package:: function** to specify which package you want to use when there is a conflict.
    * For example, filter() is in dplyr and stats. Use the syntax **dplyr::filter()**.

---

4. MARKDOWN SYNTAX

Markdown syntax applies universally across .md, .Rmd, .qmd, and .Rnw files.

- Bullet points
  - You can use either dashes (-) or asterisks (*) to create bullet points.
  - Always put a space after the symbol or the bullet will break.
  - To create a sub-bullet, press space 2 times (or hit Tab once) before typing the bullet symbol.
  - Leave a completely blank line between a regular paragraph and the start of a bulleted list so Markdown formats it correctly.

- Text Formatting
  - Bolding: Wrap in double asterisks or double underscores.
  - Italics: Wrap in single asterisks or single underscores.
  - Inline Code: Wrap in single back ticks. Looks like computer code.
  - Escape Character: Use a backslash before a symbol to display the symbol itself and not treat it as code.

- Structures & Links
  - Headers: Use hash symbols. The more hashes, the smaller the header.
    * Like bullets, you must put a space after the hash.
  - Hyperlinks: Use square brackets for the display text, followed by parentheses for the URL.
    * example: [Google](https://www.google.com)

---

5. GIT & VERSION CONTROL

Git is your ultimate safety net: it tracks changes so you can experiment with code without worrying about permanently breaking a working file.

- The Daily Workflow: Saving Work to Git

  Go to the **Git Tab** located in the Environment/History pane:

  1. **Stage:** Check the box next to the files you modified. This tells Git, *"I want to save the changes in these specific files."*
  2. **Commit (Checkbox icon):** Take a snapshot of your staged files and write a short, descriptive message (e.g., `added data cleaning script`).

  – *Tip:* Make commits small and frequent. It's much easier to undo a tiny mistake than a massive one.

  3. **Push (Upward arrow icon):** Send your local commits up to GitHub to back up your work in the cloud.

- The Life-Saving Rescue Commands

  If you make a horrible mistake or find yourself stuck in "Git Jail," open your RStudio Terminal and use these fixes:

  – **The "Undo My Last Unpushed Commit" Command:** Did you commit something by accident but haven't pushed it to GitHub yet?
    * *Command:* `git reset --soft HEAD~1` (This safely undoes the commit but keeps your typed code intact in your file so you can fix it).
  – **The Emergency Stop (Discard All Changes):** Did you completely wreck your code and just want to reset the file back to how it looked at your last successful commit?
    * *Command:* `git checkout -- filename.Rmd` (Warning: This permanently deletes your uncommitted changes in that file and restores the last saved version).
  – **The "What Did I Do?" Command:** If your Git status looks messy or confused.
    * *Command:* `git status` (This lists exactly what files are modified, staged, or untracked).

- Critical Best Practice

  – **The `.gitignore` File:** Every RStudio Project has a hidden file called `.gitignore`. Always add massive datasets (like `.csv` or `.xlsx` files over 100MB) or private API keys to this file. GitHub will block your pushes if your files are too large, and you never want to accidentally leak sensitive data online.

Why was it so hard today?

You ran into a perfect storm of setup hurdles that only happen at the very beginning:

- **The "First Connection" Setup:** You had to link the remote URL and deal with security tokens (PAT), which you only have to do once per project.
- **The Folder Mix-up:** The file started out outside the project folder, which paused Git from tracking it.
- **The GitHub Collision:** GitHub created a file online while you had one locally, causing a "merge conflict." Git was just trying to protect your work by asking you which one to keep before overwriting anything.

Think of it like building a house: today you had to pour the concrete foundation and install the plumbing. From now on, you just get to walk through the front door and turn on the lights.

Are there any other files in your project that you want to practice staging and pushing using the RStudio buttons now that the setup is done?

add README tips: Here is the standard anatomy of a great project README:

Markdown # Data Science Bootcamp

A short, one-sentence description of what this repository is for (e.g., "My central repository for tracking progress, notes, and projects during the 2026 Summer Data Science Bootcamp.")

Project Overview

A longer description where you explain what you are learning, the goals of the bootcamp, or the specific topics you are diving into (R, Python, Data Visualization, etc.).

Repository Structure

A quick breakdown of what your files are so people can navigate your repository: * `bootcamp_progress_log.txt`: My daily tracker of hours and topics covered. * `rules_and_tips.md`: A guide to RStudio workflows and best practices.

Tools & Technologies

- **Language:** R (v4.4+)
- **IDE:** RStudio / Quarto
- **Version Control:** Git & GitHub

How to Use This Repo

Brief instructions if someone wants to clone your code or look at your work. Pro-Tips for a Great README: Use Headers: Use ## for main sections and ### for subsections to keep it organized.

Use Lists: Use bullet points (* or -) for things like file structures or tool lists. It makes it much easier to read than a giant wall of text.

Keep it Living: Don't feel like it has to be perfect right now. A README is a "living document"—as you add new projects or folders over the summer, just open it up and update it.

Once you finish editing your README, save it, and it will pop right up into your Git tab just like your other file did.

What are some of the main topics or modules you'll be tackling first in the bootcamp that you want to highlight in your overview?

add how to create subdirectories on github: Option B: The Local RStudio Method (Recommended for Your Workflow)Since you already have your RStudio local environment rules ready, the most professional way to do this is on your computer:
Open your bootcamp project folder on your computer's finder/file explorer. Create your new physical folders right there (e.g., create a folder named sql-practice and r-data-science). Drag and drop your existing progress logs or text files into those new folders. Open RStudio, go to your Git tab in the top right corner. You will see your files pop up with little yellow ? or blue M icons showing they have moved.
Check the boxes to stage them, type a commit message like "reorganized repo into clean subdirectories", and hit Push!
Your main branch will instantly update, and anyone visiting your profile will see a beautifully structured, professional directory.

Clean Git Tab Rule: Having a clean, completely empty Git tab at the end of every study or work session is the gold standard of professional engineering practice.

When your Git tab is empty, it means your local environment on your computer is in perfect, 100% sync with your remote backup on GitHub.